

torch.nn.utils.parametrizations.orthogonal

<https://pytorch.org/docs/stable/generated/torch.nn.utils.parametrizations.orthogonal.html>

Description:

Apply an orthogonal or unitary parametrization to a matrix or a batch of matrices. Letting $K \in \mathbb{R}^{m \times n}$ or $\mathbb{C}^{m \times n}$, the parametrized matrix $Q \in K^{m \times n}$ is orthogonal as where $Q^H Q = I$ is the conjugate transpose when Q is complex and the transpose when Q is real-valued, and I_n is the n -dimensional identity matrix. In plain words, Q will have orthonormal columns whenever $m \geq n$ and orthonormal rows otherwise. If the tensor has more than two dimensions, we consider it as a batch of matrices of shape $(\dots, m, n)$. The matrix Q may be parametrized via three different `orthogonal_map` in terms of the original tensor:

Function Signature

```
torch.nn.utils.parametrizations.orthogonal(module,  
name='weight', orthogonal_map=None, *,  
use_trivialization=True)[source]#
```

Parameters

Returns

Return type

Returns:

Module

Code Examples

```
>>> orth_linear = orthogonal(nn.Linear(20, 40))
>>> orth_linear
ParametrizedLinear(
  in_features=20, out_features=40, bias=True
  (parametrizations): ModuleDict(
    (weight): ParametrizationList(
      (0): _Orthogonal()
    )
  )
)
>>> Q = orth_linear.weight
>>> torch.dist(Q.T @ Q, torch.eye(20))
tensor(4.9332e-07)
```

Notes & Warnings

NOTE This function is implemented using the parametrization functionality in `register_parametrization()`.

Detailed Documentation

Documentation

Apply an orthogonal or unitary parametrization to a matrix or a batch of matrices.

Letting $K \in \mathbb{R} \text{ or } \mathbb{C}$, the parametrized matrix $Q \in K^{m \times n}$ is orthogonal as

where $Q^H Q$ is the conjugate transpose when Q is complex and the transpose when Q is real-valued, and I_n is the n -dimensional identity matrix. In plain words, Q will have orthonormal columns whenever $m \geq n$ and orthonormal rows otherwise.

If the tensor has more than two dimensions, we consider it as a batch of matrices of shape $(\dots, m, n)$.

The matrix Q may be parametrized via three different orthogonal_map in terms of the original tensor:

"matrix_exp"/"cayley": the matrix_exp() $Q = \exp(A)$ and the Cayley map $Q = (I_n + A/2)(I_n - A/2)^{-1}$ are applied to a skew-symmetric A to give an orthogonal matrix.

"householder": computes a product of Householder reflectors (householder_product()).

"matrix_exp"/"cayley" often make the parametrized weight converge faster than "householder", but they are slower to compute for very thin or very wide matrices.

If use_trivialization=True (default), the parametrization implements the “Dynamic Trivialization Framework”, where an extra matrix $B \in K^{n \times n}$ is stored under module.parametrizations.weight[0].base. This helps the convergence of the parametrized layer at the expense of some extra memory use. See Trivializations for Gradient-Based Optimization on Manifolds .

Initial value of Q : If the original tensor is not parametrized and use_trivialization=True (default), the initial value of Q is that of the original tensor if it is orthogonal (or unitary in the complex case) and it is orthogonalized via the QR decomposition

otherwise (see `torch.linalg.qr()`). Same happens when it is not parametrized and `orthogonal_map="householder"` even when `use_trivialization=False`. Otherwise, the initial value is the result of the composition of all the registered parametrizations applied to the original tensor.

This function is implemented using the parametrization functionality in `register_parametrization()`.

`module (nn.Module)` – module on which to register the parametrization.

`name (str, optional)` – name of the tensor to make orthogonal. Default: "weight".

`orthogonal_map (str, optional)` – One of the following: "matrix_exp", "cayley", "householder". Default: "matrix_exp" if the matrix is square or complex, "householder" otherwise.

`use_trivialization (bool, optional)` – whether to use the dynamic trivialization framework. Default: True.

The original module with an orthogonal parametrization registered to the specified weight